

CS CollabSphere

A Collaboration Marketplace for Creators, Influencers & Brands

System Design & Software Engineering (SDSE)

Project Report — Sprint 1 Implementation

Repository: github.com/samiksha-jangid27/CollabSphere

Live Demo: collab-sphere-amber.vercel.app

Stack: Next.js 15 · Express.js 5 · MongoDB Atlas · TypeScript

Date: May 2026

Team Members

Samiksha Jangid

(add remaining team member names)

Submitted in partial fulfilment of the SDSE course requirements

Contents

1	Project Overview	2
1.1	Problem Statement	2
1.2	Solution Approach	2
1.3	Tech Stack Summary	3
2	System Design & Architecture	4
2.1	Architectural Overview	4
2.1.1	Three-Layer Backend Architecture	4
2.2	Scalability Considerations	5
2.3	API Design	5
3	Object-Oriented Programming Concepts	7
3.1	Encapsulation	7
3.2	Abstraction	7
3.3	Inheritance	8
3.4	Polymorphism	9
4	Design Patterns	10
4.1	Observer Pattern	10
4.2	Repository Pattern	11
4.3	Additional Patterns	12
5	SOLID Principles	13
5.1	S — Single Responsibility Principle	13
5.2	O — Open/Closed Principle	13
5.3	L — Liskov Substitution Principle	14
5.4	I — Interface Segregation Principle	14
5.5	D — Dependency Inversion Principle	14
6	UML Diagrams	16
6.1	Use Case Diagram	16
6.2	Class Diagram	17
6.3	Entity-Relationship (ER) Diagram	18
6.4	Sequence Diagrams	18
6.4.1	User Sign-Up & Verification	18

6.4.2	Location-Based Search	19
6.4.3	Collaboration Request Flow (Observer Pattern)	19
6.4.4	Social Account Linking (OAuth 2.0)	19
7	SDLC & Development Process	20
7.1	Development Model	20
7.2	Documentation Artefacts	20
7.3	Version Control Strategy	20
8	Test Cases & Results	22
8.1	Testing Strategy	22
8.2	Test Results	22
8.2.1	User Model Test Cases (13 tests)	22
8.2.2	Auth Integration Test Cases (11 tests)	23
8.3	Running the Tests	23
9	Setup & Installation	25
9.1	Prerequisites	25
9.2	Installation Steps	25
9.3	Development Login Flow	26
10	Conclusion	27
10.1	Summary	27
10.2	Key Achievements	27
10.3	Future Work	27
A	Project Structure Reference	29
B	Environment Variables Reference	31

Chapter 1

Project Overview

1.1 Problem Statement

The creator economy is growing rapidly, yet the process of forging collaborations between influencers, brands, and fellow creators remains fragmented and manual. Creators juggle cold DMs across multiple social platforms, brands lack a structured way to discover niche-relevant influencers, and there is no single trusted channel where verified credentials, contact details, and collaboration intent co-exist.

CollabSphere addresses this gap by providing a dedicated collaboration marketplace where:

- Creators build verified, portfolio-rich profiles linked to their real social accounts.
- Brands post structured collaboration opportunities and browse applicant pools.
- All parties communicate through an in-platform secure messaging layer.
- Discovery is powered by geospatial search, niche filtering, and verified-first ranking.

1.2 Solution Approach

The project adopts an incremental, sprint-based delivery model aligned with the Software Development Life Cycle (SDLC). Sprint 1 — the subject of this report — delivers the authentication infrastructure and core data model. Subsequent sprints layer in profiles, collaboration requests, messaging, and moderation.

The backend is built as a modular **Express.js 5** application with TypeScript, following a strict three-layer architecture (Controller → Service → Repository). The frontend is a **Next.js 15** application with server-side rendering. Persistence is provided by **MongoDB Atlas** with Mongoose ODM.

1.3 Tech Stack Summary

Table 1.1: CollabSphere Technology Stack

Layer	Technology
Frontend	Next.js 15, React 19, Tailwind CSS v4, TypeScript
Backend	Node.js v23+, Express.js 5, TypeScript
Database	MongoDB Atlas (cloud), Mongoose ODM v9
Authentication	JWT (access + refresh + email tokens), Phone OTP, Email verification
OTP Delivery	Twilio (production) / terminal console (development)
Email Delivery	Nodemailer + Ethereal (development) / SMTP (production)
Validation	Zod (schema-first, server-side)
Logging	Winston
Testing	Jest, Supertest, mongodb-memory-server
Dev Tooling	concurrently, tsx, ts-node

Chapter 2

System Design & Architecture

2.1 Architectural Overview

CollabSphere follows a **client-server architecture** with a clear separation of concerns across four distinct tiers:

1. **Presentation Tier** — Next.js frontend served on port 3000. Responsible for rendering, routing, and user interaction. Protected routes rely on JWTs stored in HTTP-only cookies managed by an `AuthContext`.
2. **API Tier** — Express.js REST API served on port 5001. Accepts JSON requests, validates them via Zod middleware, authenticates via JWT middleware, and delegates to service modules.
3. **Business Logic Tier** — Modular service classes encapsulate all business rules, keeping controllers thin and testable.
4. **Data Tier** — MongoDB Atlas accessed through Mongoose models and a generic `BaseRepository` abstraction.

2.1.1 Three-Layer Backend Architecture

Request Lifecycle

Incoming HTTP Request $\xrightarrow{\text{Middleware stack}}$ **Controller** (input parsing + response shaping) $\rightarrow$ **Service** (business logic) $\rightarrow$ **Repository** (database queries) $\rightarrow$ **MongoDB Atlas**

This layering achieves:

- *Testability* — services can be unit-tested by mocking repositories.
- *Replaceability* — swapping MongoDB for PostgreSQL only requires new repository implementations.
- *Readability* — each file has a single, clearly named responsibility.

2.2 Scalability Considerations

- **Stateless API:** JWT-based authentication allows horizontal scaling of the backend without shared session state.
- **MongoDB Atlas:** Managed cloud database with auto-scaling and multi-region replication support.
- **Geospatial indexing:** A `2dsphere` index on `Profile.location` enables efficient *\$near* queries without full-collection scans.
- **Rate limiting:** `rateLimiter.ts` middleware prevents OTP and auth endpoint abuse.
- **Event-driven notifications:** The `EventBus` (Observer pattern) decouples real-time notification delivery from core business logic, allowing independent scaling of the notification subsystem.

2.3 API Design

The REST API follows conventional HTTP semantics (GET, POST, PATCH, DELETE), versioned under `/api/v1`, with a consistent JSON envelope:

Listing 2.1: Standard API Response Envelope

```
1 {  
2   "success": true,  
3   "message": "OTP verified successfully",  
4   "data": {  
5     "user": { "id": "...", "phone": "+91..." },  
6     "accessToken": "eyJ..."  
7   }  
8 }
```

Sprint 1 API Endpoints:

Table 2.1: Implemented REST Endpoints

Method	Path	Auth	Description
POST	/auth/otp/send	Public	Send OTP to phone number
POST	/auth/otp/verify	Public	Verify OTP, issue JWT tokens
POST	/auth/email/send	Bearer	Send verification email
GET	/auth/email/verify/:token	Public	Confirm email via signed link
POST	/auth/refresh	Cookie	Rotate access + refresh tokens
POST	/auth/logout	Bearer	Invalidate session
GET	/auth/me	Bearer	Retrieve current user
GET	/health	Public	Server health check

Chapter 3

Object-Oriented Programming Concepts

OOP forms the backbone of CollabSphere’s backend design. Each of the four core pillars is applied deliberately and traceable to specific files.

3.1 Encapsulation

Encapsulation is enforced by hiding implementation details behind well-defined interfaces.

File: `server/src/modules/auth/auth.service.ts`

The `AuthService` class owns the OTP generation, hashing, expiry logic, and JWT orchestration internally. Controllers never access the `User` model or token utilities directly — they call public methods like `sendOtp()` and `verifyOtp()`.

Listing 3.1: Service encapsulates business logic

```
1 // Controller only calls the public interface:
2 const result = await authService.verifyOtp({ phone, otp
  });
3 // All internal steps (hash comparison, expiry check,
4 // token generation) are hidden inside the service.
```

Similarly, `BaseRepository.ts` encapsulates generic MongoDB CRUD operations. Each domain repository (e.g. `auth.repository.ts`) inherits these without exposing the Mongoose query API to the service layer.

3.2 Abstraction

Abstraction is achieved by programming to interfaces and abstract base classes.

File: server/src/shared/BaseRepository.ts

`BaseRepository<T>` is a generic abstract class that defines `findById`, `create`, `update`, and `delete` signatures. Concrete repositories implement domain-specific queries, while services depend only on the abstracted contract.

Listing 3.2: Abstract repository base

```
1 abstract class BaseRepository<T> {
2   abstract findById(id: string): Promise<T | null>;
3   abstract create(data: Partial<T>): Promise<T>;
4   abstract update(id: string, data: Partial<T>):
      Promise<T | null>;
5   abstract delete(id: string): Promise<boolean>;
6 }
```

TypeScript interfaces in `auth.interfaces.ts` define the shape of DTOs (Data Transfer Objects) such as `SendOtpDto` and `VerifyOtpDto`, providing compile-time abstraction over raw request payloads.

3.3 Inheritance

Inheritance in Action

File: server/src/shared/errors.ts

`AppError` extends the built-in `Error` class to carry HTTP status codes and machine-readable error codes. Specialised error subtypes (`ValidationError`, `AuthenticationError`, `NotFoundError`) inherit from `AppError`, enabling the global error handler to treat them uniformly.

Listing 3.3: Error hierarchy via inheritance

```
1 class AppError extends Error {
2   constructor(public message: string,
3               public statusCode: number,
4               public code: string) { super(message); }
5 }
6 class AuthenticationError extends AppError {
7   constructor(msg: string) {
8     super(msg, 401, 'AUTHENTICATION_ERROR');
9   }
}
```

```
10 }
```

Domain repositories also inherit `BaseRepository`, gaining shared CRUD behaviour while adding specialised queries (e.g. `findByPhone`, `findByOtpToken`).

3.4 Polymorphism

Files: `otp.provider.ts`, `email.provider.ts`

Both providers implement a shared interface but behave differently depending on the environment. `OtpProvider` either logs the OTP to the terminal (development) or dispatches it via the Twilio SDK (production) — the calling service issues the same `send(phone, otp)` call regardless.

Listing 3.4: Provider polymorphism

```
1 interface IOtpProvider { send(phone: string, otp:
   string): Promise<void>; }
2
3 class ConsoleOtpProvider implements IOtpProvider {
4   async send(phone, otp) { logger.info('[DEV] OTP:
     ${otp}'); }
5 }
6 class TwilioOtpProvider implements IOtpProvider {
7   async send(phone, otp) { await
     twilioClient.messages.create(...); }
8 }
9 // Runtime selection based on environment variables
```

The global error handler in `errorHandler.ts` also demonstrates polymorphism: it calls `instanceof` checks to handle `AppError`, `Mongoose ValidationError`, and `CastError` differently, producing appropriately shaped error responses.

Chapter 4

Design Patterns

CollabSphere implements several recognised GoF and architectural design patterns. The two primary patterns are documented in detail below.

4.1 Observer Pattern

File: `server/src/shared/EventBus.ts`

Intent: Define a one-to-many dependency so that when one object changes state, all its dependents are notified automatically.

Application: The `EventBus` singleton acts as the *Subject*. When a collaboration request is created or accepted, the service emits a typed event (`collab.created`, `collab.accepted`). Subscribed listeners — the `Socket.io` notification handler and the conversation-creation handler — react independently without the service knowing about them.

Participants:

- *Subject* — `EventBus`: maintains a list of event listeners and emits events.
- *Concrete Observers* — `Socket.io` notification handler, Conversation auto-creation handler.
- *Event* — typed string keys such as `'collab.created'` with a payload interface.

Benefit: Decouples collaboration business logic from notification and conversation concerns. Adding a new observer (e.g. an email digest handler) requires zero changes to `CollaborationService`.

4.2 Repository Pattern

Files: `BaseRepository.ts`, `auth.repository.ts`

Intent: Mediate between the domain and data-mapping layers using a collection-like interface for accessing domain objects.

Application: `BaseRepository<T>` defines the generic CRUD interface. `AuthRepository` extends it and adds domain-specific finders (`findByPhone`, `upsertByPhone`). Services receive `AuthRepository` via constructor injection and call only its public API, never issuing Mongoose queries directly.

Benefit: Centralises data access logic, makes services trivially unit-testable with mock repositories, and insulates the codebase from future database migrations.

4.3 Additional Patterns

Table 4.1: Additional Design Patterns Applied

Pattern	Location	How it is Applied
Strategy	<i>otp.provider.ts / email.provider.ts</i>	Runtime selection between console and Twilio/SMTP providers based on environment.
Singleton	<i>EventBus.ts, database.ts</i>	EventBus and MongoDB connection are singleton instances shared across the application.
Middleware Chain	<i>middleware/</i>	Express middleware

Chapter 5

SOLID Principles

5.1 S — Single Responsibility Principle

Definition

A class should have only one reason to change.

Every module in CollabSphere is scoped to a single concern:

- `auth.controller.ts` — parses HTTP requests and shapes responses only.
- `auth.service.ts` — applies business rules for authentication only.
- `auth.repository.ts` — executes database queries for the `User` collection only.
- `token.service.ts` — generates and verifies JWTs only.
- `otp.provider.ts` — delivers OTPs only.
- `logger.ts` — wraps Winston logging configuration only.

5.2 O — Open/Closed Principle

Definition

Software entities should be open for extension but closed for modification.

The provider abstraction (`IOTPProvider`, `IEmailProvider`) exemplifies OCP. Adding a WhatsApp OTP provider requires only creating a new class that implements `IOTPProvider` and registering it in the factory — the `AuthService` code does not change. Likewise, the `EventBus` allows new event listeners to be added without touching existing subscribers or the emitter.

5.3 L — Liskov Substitution Principle

Definition

Objects of a superclass should be replaceable with objects of its subclasses without affecting correctness.

- `TwilioOtpProvider` and `ConsoleOtpProvider` both satisfy `IOtpProvider` and are transparently interchangeable in `AuthService`.
- `AuthRepository` extends `BaseRepository` and is used polymorphically wherever a `BaseRepository<User>` is expected.
- All `AppError` subclasses are safely caught by the global error handler, which expects an `AppError` instance.

5.4 I — Interface Segregation Principle

Definition

No client should be forced to depend on interfaces it does not use.

The auth module exposes narrow TypeScript interfaces per concern:

- `IAuthService` — login/logout/refresh operations only.
- `ITokenService` — token generation and verification only.
- `IOtpProvider` — `send(phone, otp)` only.
- `IEmailProvider` — `sendVerification(email, token)` only.

Consumers depend on the minimal interface that satisfies their need, rather than a monolithic service facade.

5.5 D — Dependency Inversion Principle

Definition

High-level modules should not depend on low-level modules. Both should depend on abstractions.

`AuthService` receives its dependencies (`AuthRepository`, `OtpProvider`, `TokenService`) through constructor injection. The concrete implementations are wired in the application bootstrap (`index.ts`). This means:

- Unit tests inject mock implementations without touching production code.
- Switching from Twilio to a different SMS gateway requires no change inside `AuthService`.

- Zod-validated environment configuration (`environment.ts`) centralises all external dependency configuration, validated at startup.

Chapter 6

UML Diagrams

The UML diagrams described in this chapter were developed using Mermaid and are version-controlled in `UML Designs/mermaid/` within the repository, keeping the design in sync with the implementation.

6.1 Use Case Diagram

Purpose: Captures all actors and system capabilities at a high level.

Actors:

- **Creator / Influencer** — Primary user; builds a profile, searches for collaborations, and communicates with brands.
- **Brand** — Secondary user; posts opportunities, browses creators, manages campaign requests.
- **Admin / Moderator** — Platform operator; moderates content, manages verifications, bans users.

Use Case Groups:

Table 6.1: Use Case Groupings

Group	Use Cases
Onboarding & Profile	Sign Up with Phone OTP, Verify Email, Create Profile, Edit Profile, Link Social Accounts, Submit Verification Request
Discovery & Search	Search Creators, Filter Results, View Profiles, Bookmark Profiles
Interaction & Collaboration	Send Collaboration Request, Accept/Reject Request, Send Messages, View Messages
Brand Operations	Post Collaboration Opportunity, Browse Applicants, Manage Campaign Requests
Moderation	Moderate Reported Content, View Statistics, Ban/-Suspend Users, Approve/Reject Profiles
System Services	Send OTP via SMS, Send Email Link, Validate JWT, Execute Geospatial Queries, Deliver Real-Time Messages

Dotted <<include>> relationships connect primary use cases to internal system services (e.g. Sign Up includes Send OTP via SMS).

6.2 Class Diagram

Purpose: Documents the OOP structure of the backend — controllers, services, domain models, and their relationships.

Key classes and relationships:

- `UserController` → `UserService`, `AuthService` (delegation)
- `CollaborationRequestController` → `CollaborationService` (delegation)
- `User` ↔ `Profile` (one-to-one composition)
- `Profile` ↔ `SocialAccount`, `MediaPost` (one-to-many)
- `Conversation` ↔ `Message` (one-to-many)
- `VerificationRequest` → `VerificationStatus` (enumeration)
- `CollaborationRequest` → `RequestStatus` (enumeration)

Enumerations defined: `UserRole` (CREATOR, BRAND, ADMIN), `VerificationStatus` (PENDING, APPROVED, REJECTED), `RequestStatus` (PENDING, ACCEPTED, REJECTED, CLOSED), `MessageType` (TEXT, MEDIA).

6.3 Entity-Relationship (ER) Diagram

Purpose: Models the MongoDB document schema and cardinalities.

Entities and cardinalities:

Table 6.2: ER Cardinalities

Entity A	Relationship	Entity B
USER	exactly-one $\longleftrightarrow$ exactly-one	PROFILE
PROFILE	one-to-many	SOCIAL_ACCOUNT
PROFILE	one-to-many	MEDIA_POST
USER	one-to-many	COLLABORATION_REQUEST
USER	one-to-many	CONVERSATION
CONVERSATION	one-to-many	MESSAGE
USER	one-to-many	BOOKMARK
USER	one-to-many	VERIFICATION_REQUEST
USER	one-to-many	REPORT

Notable design decision: Social accounts and media posts are embedded as arrays within the Profile document, leveraging MongoDB's document model for co-locality and query efficiency.

6.4 Sequence Diagrams

6.4.1 User Sign-Up & Verification

Flow Summary:

1. User submits phone number; Frontend posts to `/auth/otp/send`.
2. Backend dispatches OTP via Twilio (or terminal in dev).
3. User submits the 6-digit OTP; Frontend posts to `/auth/otp/verify`.
4. Backend locates or creates a User document in MongoDB.
5. Backend generates a signed JWT access token and a rotating refresh token; returns both to the Frontend.
6. From the dashboard, user requests email verification; Backend sends a signed email link via Nodemailer.
7. User clicks the link; Backend validates the email token and marks `emailVerified = true` in MongoDB.

6.4.2 Location-Based Search

Flow Summary:

1. User sets geographic filters (latitude, longitude, radius) and niche/platform filters.
2. Frontend issues `GET /search/nearby?lat=&lng=&radius=`.
3. Backend executes a `$near` geospatial query against the `2dsphere` index on `Profile.location`.
4. Post-query niche and platform filters are applied in-memory.
5. Results are paginated and returned as a profile card array.

6.4.3 Collaboration Request Flow (Observer Pattern)

Flow Summary:

1. Sender submits a collaboration request; Backend validates and creates a record in MongoDB.
2. `CollaborationService` emits a `collab.created` event on `EventBus`.
3. The `Socket.io` observer delivers a real-time notification to the Receiver's frontend.
4. Receiver opens and accepts the request; Backend patches the status to `ACCEPTED`.
5. `collab.accepted` event triggers automatic `Conversation` creation for the two parties.

6.4.4 Social Account Linking (OAuth 2.0)

Flow Summary:

1. User clicks "Link Instagram"; Frontend posts to `/socials/oauth/instagram`.
2. Backend returns an OAuth redirect URL for the provider.
3. User grants permission; Provider calls back to Backend with an authorisation code.
4. Backend exchanges the code for an access token, retrieves the user's handle, and verifies it matches the claimed profile.
5. Backend persists the `SocialAccount` document with `isVerified = true`.

Chapter 7

SDLC & Development Process

7.1 Development Model

CollabSphere follows an **Agile sprint model** with vertical slices of functionality delivered each sprint:

Table 7.1: Sprint Delivery Plan

Sprint	Status	Deliverables
1	✔ Complete	Auth system (Phone OTP, Email verification, JWT rotation), User model, middleware stack, test suite
2	Planned	Profile creation, social account linking, geospatial search
3	Planned	Collaboration request workflow, real-time notifications (Socket.io)
4	Planned	Messaging, bookmarks, admin moderation, reporting

7.2 Documentation Artefacts

Sprint 1 produced four formal documentation files in `docs/`:

- **PRD.md** — Product requirements, user stories, acceptance criteria.
- **API_SPEC.md** — Complete endpoint specification with request/response examples.
- **DATA_MODEL.md** — MongoDB schema design and indexing strategy.
- **AUTH_FLOWS.md** — Step-by-step authentication flow diagrams.

7.3 Version Control Strategy

The project uses Git with a feature-branch workflow hosted on GitHub. UML Mermaid sources are committed alongside code, ensuring design artefacts remain in sync with the

implementation at every commit.

Chapter 8

Test Cases & Results

8.1 Testing Strategy

CollabSphere employs a two-level testing strategy:

1. **Unit / Model Tests** (`user.model.test.ts`) — validate Mongoose schema constraints, OTP subdocument behaviour, virtual fields, and static methods in isolation.
2. **Integration Tests** (`auth.integration.test.ts`) — spin up the full Express application against an in-memory MongoDB instance (`mongodb-memory-server`) and exercise every auth endpoint via HTTP using Supertest.

No external services are required for testing: Twilio is replaced by a console stub and MongoDB is fully in-memory, making the test suite runnable on any developer machine or CI runner with zero provisioning.

8.2 Test Results

Table 8.1: Sprint 1 Test Results

Suite	File	Tests	Passed	Status
User Model Tests	<code>user.model.test.ts</code>	13	13	✓ PASS
Auth Integration Tests	<code>auth.integration.test.ts</code>	11	11	✓ PASS
Total		24	24	✓ ALL PASS

8.2.1 User Model Test Cases (13 tests)

1. Should create a user with a valid phone number.
2. Should reject a user without a phone number.
3. Should reject a duplicate phone number (unique index).
4. Should default `phoneVerified` to `false`.
5. Should default `emailVerified` to `false`.

6. Should default `role` to `CREATOR`.
7. Should accept all valid roles (`CREATOR`, `BRAND`, `ADMIN`).
8. Should reject an invalid role value.
9. Should store an OTP subdocument correctly.
10. Should enforce OTP expiry field.
11. Should store email and update `emailVerified`.
12. Should allow `deactivate()` to set `isActive` = `false`.
13. Should return correct `id` virtual field.

8.2.2 Auth Integration Test Cases (11 tests)

1. `POST /otp/send` — returns 200 and sends OTP for a valid phone number.
2. `POST /otp/send` — returns 400 for a missing phone number.
3. `POST /otp/send` — returns 400 for an invalid phone format.
4. `POST /otp/send` — returns 429 after rate limit is exceeded.
5. `POST /otp/verify` — returns 200 and JWT tokens for a valid OTP.
6. `POST /otp/verify` — returns 401 for an incorrect OTP.
7. `POST /otp/verify` — returns 401 for an expired OTP.
8. `GET /me` — returns current user for a valid Bearer token.
9. `GET /me` — returns 401 for a missing token.
10. `POST /refresh` — rotates tokens successfully with a valid refresh cookie.
11. `POST /logout` — invalidates the session and clears cookies.

8.3 Running the Tests

Listing 8.1: Running the full test suite

```
1 # From project root:
2 npm test
3
4 # Expected output:
5 PASS tests/auth/user.model.test.ts (13 tests)
6 PASS tests/auth/auth.integration.test.ts (11 tests)
7
8 Test Suites: 2 passed, 2 total
```

```
9 Tests:      24 passed, 24 total
10 Time:      ~8s
```

Chapter 9

Setup & Installation

9.1 Prerequisites

- Node.js v23 or higher (`node -v` to check; `nvm use` reads `.nvmrc`).
- npm (bundled with Node.js).
- Git.
- A MongoDB Atlas connection string (obtain from the team lead).

Not required for development: Twilio account, Gmail / SMTP credentials, or Docker.

9.2 Installation Steps

Listing 9.1: Step-by-step setup

```
1  # 1. Clone the repository
2  git clone git@github.com:samiksha-jangid27/CollabSphere.git
3  cd CollabSphere
4
5  # 2. Install dependencies (three directories)
6  npm install
7  cd server && npm install && cd ..
8  cd client && npm install && cd ..
9
10 # 3. Configure environment variables
11 cp server/.env.example server/.env
12 # Open server/.env and set:
13 #
14     MONGODB_URI=mongodb+srv://<user>:<pass>@<cluster>.mongodb.net/collabsphe
15
16 # 4. Start the application
17 npm run dev
18 # Backend: http://localhost:5001
19 # Frontend: http://localhost:3000
```

9.3 Development Login Flow

1. Open `http://localhost:3000/login` in your browser.
2. Enter any phone number (e.g. 9876543210) and click **Send OTP**.
3. Find the 6-digit code in the server terminal output: `[info]: [DEV] OTP for +91...: XXXXXX`.
4. Enter the code in the browser and click **Verify** — you are redirected to the dashboard.

Chapter 10

Conclusion

10.1 Summary

CollabSphere demonstrates a production-grade full-stack application built from the ground up with sound software engineering principles. Sprint 1 delivers a complete, tested authentication system that serves as the secure foundation for all future feature sprints.

The project applies all four OOP pillars concretely and traceably, implements at least six recognised design patterns (Observer, Repository, Strategy, Singleton, Middleware Chain, DTO), and satisfies every SOLID principle in the server codebase. Seven UML diagrams document the system design at multiple levels of abstraction.

10.2 Key Achievements

- **24 / 24 tests passing** across model and integration suites.
- **Zero external services required** for development or testing.
- **Layered architecture** enabling independent scaling and easy testability.
- **Type-safe end-to-end** — TypeScript on both client and server, Zod runtime validation.
- **Event-driven design** — Observer pattern decouples real-time notifications from business logic.
- **Living documentation** — UML sources versioned alongside code.

10.3 Future Work

- **Sprint 2:** Creator profile pages, social account OAuth linking, geospatial creator discovery.
- **Sprint 3:** Collaboration request workflow with real-time Socket.io notifications.
- **Sprint 4:** In-platform messaging, bookmarks, admin dashboard, and content moderation.

- **Production hardening:** CI/CD pipeline, Docker containerisation, environment-specific JWT secrets rotation, Redis-backed rate limiting and session store.

Appendix A

Project Structure Reference

Listing A.1: Annotated Repository Structure

```
1 CollabSphere/
2 |-- client/                                # Next.js 15 frontend
3 |   |-- src/app/
4 |   |   |-- (auth)/login/page.tsx         # Phone + OTP login page
5 |   |   |-- (auth)/verify/page.tsx        # Email verification page
6 |   |   |-- layout.tsx                    # Root layout (dark theme)
7 |   |   |-- page.tsx                      # Protected dashboard
8 |   |-- components/ui/                    # Button, Input, OtpInput,
      Card
9 |   |-- context/AuthContext.tsx           # Auth state + silent JWT
      refresh
10 |   |-- services/authService.ts          # Auth API calls via Axios
11 |
12 |-- server/                               # Express.js 5 backend
13 |   |-- src/
14 |   |   |-- config/                      # Zod-validated env + DB
      connection
15 |   |   |-- middleware/                  # JWT auth, RBAC, Zod
      validate, rate limit
16 |   |   |-- models/User.ts              # User schema + OTP
      subdocument
17 |   |   |-- modules/auth/                # Controller, Service,
      Repository,
18 |   |   |   |                            # Routes, Validation,
      Interfaces,
19 |   |   |   |                            # TokenService,
      OtpProvider, EmailProvider
20 |   |   |-- shared/                      # BaseRepository, EventBus,
      errors,
21 |   |   |   |                            # responseHelper,
      constants, logger
```

```
22 |-- tests/                                # Jest test suites
23 |-- docs/                                # PRD, API_SPEC,
    DATA_MODEL, AUTH_FLOWS
24 '-- UML Designs/mermaid/                # Version-controlled
    diagram sources
```

Appendix B

Environment Variables Reference

Table B.1: Server Environment Variables

Variable	Required	Description
MONGODB_URI	Yes	MongoDB Atlas connection string
PORT	No (5001)	Backend server port
NODE_ENV	No (development)	Runtime environment mode
JWT_ACCESS_SECRET	Yes	Signing key for access tokens
JWT_REFRESH_SECRET	Yes	Signing key for refresh tokens
JWT_EMAIL_SECRET	Yes	Signing key for email tokens
JWT_ACCESS_EXPIRY	No (15m)	Access token time-to-live
JWT_REFRESH_EXPIRY	No (7d)	Refresh token time-to-live
JWT_EMAIL_EXPIRY	No (24h)	Email verification link TTL
TWILIO_ACCOUNT_SID	No	Twilio SID (dev: leave empty)
TWILIO_AUTH_TOKEN	No	Twilio auth token (dev: leave empty)
TWILIO_PHONE_NUMBER	No	Twilio sender number
SMTP_HOST	No	SMTP host (dev: leave empty for Ethe- real)
SMTP_PORT	No (587)	SMTP port
SMTP_USER	No	SMTP username
SMTP_PASS	No	SMTP password
CLIENT_URL	No	Frontend URL for CORS (default: local- host:3000)